u4lib: An AI-Enabled Computational Framework for U(4) Supermultiplet Symmetry, Wigner Coupling, and Racah Recoupling Coefficients

Phong Dang,[1] Daniel Langr,[2] and Tomas Dytrych[3,1]

[1]*Department of Physics and Astronomy, Louisiana State University, Baton Rouge, LA 70803, USA*
[2]*Department of Computer Systems, Faculty of Information Technology, Czech Technical University in Prague, Prague, Czech Republic*
[3]*Nuclear Physics Institute, Czech Academy of Sciences, 250 68 Řež, Czech Republic*

u4lib is an AI-enabled C++ framework for U(4)/SU(4) representation-theory computations — Wigner's supermultiplet symmetry — in nuclear structure and related applications. Besides the fast and accurate evaluation of U(4) Wigner coupling and Racah recoupling coefficients, implemented with the procedure of Dang *et al.* [1], the library exposes the full algebraic toolkit around those coefficients: irreducible representations (irreps) and their dimensions, Casimir invariants and conjugation; the canonical Gelfand–Tsetlin basis and the physical SU(2) × SU(2) spin-isospin basis, with their inner and outer multiplicities; the matrix elements of the U(4) generators on canonical states; the reduced matrix elements of the physical generators **S**, **T**, and the Gamow–Teller-type tensor **U**; the tensor-product decomposition; and the transformation between the physical and canonical bases. u4lib layers on u3lib (SU(3)) and mirrors its design: the C++ API is header-only and mirrored one-to-one by Python bindings, and a repository-local catalog of agent skills lets a code-executing AI assistant drive any capability from a natural-language request. This document is the reference manual for the public release.

PROGRAM SUMMARY

Program title: u4lib
Licensing provisions: MIT License.
Code repository: https://github.com/Deep-SANE/u4lib
Programming language: C++ (C++20), with Python bindings via pybind11.
External libraries: u3lib (prerequisite, providing Eigen3, Boost headers, OpenMP, fmt, libwignernj, and core utilities); and a null-space solver backend — dense Eigen3 QR (the default) or sns. The Python extension additionally uses pybind11.
Nature of problem: Fast, accurate, and reproducible evaluation of the Wigner coupling coefficients for the two group–subgroup chains SU(4) ⊃ U(3) (canonical) and SU(4) ⊃ SU(2) × SU(2) (physical, spin-isospin), the U(4) Racah recoupling coefficients (*U*-, *Z*-, and 9-coefficients), and the associated representation-theoretic quantities (dimensions, Casimirs, multiplicities, basis enumeration, generator matrix elements, and basis transformations) needed by symmetry-adapted nuclear-structure calculations.
Solution method: The coupling and recoupling coefficients are built from the highest-weight canonical reduced coefficients, obtained as the orthonormal null space of the linear system enforced by the raising generators, and propagated by the recursion relations of Ref. [1]; the physical SU(2) × SU(2)-coupled coefficients follow from the physical → canonical basis transformation.

I. INTRODUCTION

u4lib evaluates the coupling and recoupling coefficients of the unitary group U(4) and its special unitary subgroup SU(4), together with the representation theory on which they rest. SU(4) is Wigner's supermultiplet symmetry, combining spin and isospin; its coefficients are consumed in large numbers by symmetry-adapted nuclear-structure calculations, where speed, accuracy, and a stable, documented set of phase and normalization conventions are essential.

The coefficients are computed with the procedure introduced in Ref. [1]. The mathematical formulation, definitions, and governing equations are given in that paper and are not reproduced here; this manual is a reference for the software. u4lib packages the engine as a reusable framework: in addition to the coefficients it provides irreps and their invariants, the canonical (Gelfand–Tsetlin) and physical (SU(2) × SU(2)-coupled) bases, the generator matrix elements, the tensor-product decomposition, and the physical → canonical transformation. u4lib layers on u3lib and mirrors its design; the C++ API is header-only, Python bindings mirror it one-to-one, and an agent-skill catalog exposes every capability to an AI assistant. The remainder documents the repository structure (Sec. II), the classes and interface (Sec. III), the Python bindings (Sec. IV), the AI-enabled interface (Sec. V), and the build (Sec. VI).

II. LIBRARY STRUCTURE

`u4lib` is a header-only C++ library with a thin build layer; it mirrors `u3lib` and requires it as a prerequisite. The top-level directories are:

- `include/` — the library itself (all algorithms live here; user programs include these headers directly):
  - `U4irrep.h` — the irrep classes `su4::irrep`, `u4::irrep`;
  - `U4gelfand.h`, `internal/gelfand/` — the Gelfand state class `u4::gelfand`;
  - `U4basis.h` — basis enumeration (`gelfandBasis`, `physicalBasis`), the inner multiplicity `Zeta`, the `u4::physicalvector` class, and the generator-action predicates;
  - `U4decomposition.h` — the outer multiplicity `Eta` and the tensor-product `decomposition`;
  - `U4U3xU4U3.h`, `U4SU2SU2xU4SU2SU2.h` — the canonical and physical coupling coefficients and their databases;
  - `U4_U6Z6Coeffs.h`, `U4_U9Coeffs.h` — the U-, Z-, and 9-coefficient recoupling coefficients and their databases;
  - `U4SU2SU2_to_U4U3U2U1.h` — the physical $\to$ canonical transformation;
  - `U4generatorsRME.h` — the physical-generator reduced matrix elements `RME_S`, `RME_T`, `RME_U`;
  - `Definitions.h` — the structured coefficient containers;
  - `u4lib.h` — the umbrella header and the `initialize`/`finalize` lifecycle.
- `src/` — small example driver programs (one `.cpp` per task) that call the headers and print results; compiled to `bin/`.
- `python/` — the pybind11 extension (`pyu4lib.cpp` $\to$ `u4lib.so`), its build (`make python`, `build.sh`), build notes (`BUILD_NOTES.md`), and a tutorial notebook (`tutorial.ipynb`).
- `agents/` — the AI-agent skill catalog: an index (`AGENTS.md`), shared runtime instructions, and one skill per capability.
- `test/` — the regression suite (`test_*.cpp`, the `*_cases.h` case sets, and `reference/` documented values), run by `make test`.
- `doc/` — this manual (LaTeX source).
- `bin/` — build output (not source).

III. CLASSES AND LIBRARY INTERFACE

This section documents every public class and its methods, and the database/container interface through which coefficients are reached. All types are in the namespace `u4` (SU(4)-only base members in `su4`). The algebraic types and their methods do not require initialization; the coefficient *databases* do. Table I is an at-a-glance map of the main entry points.

A. Lifecycle

In `u4lib.h`:

- `initialize()` / `initialize(&argc,&argv)` — set up the Wigner backend, the SO(3) caches, and the U(3) *and* U(4) coefficient databases (the cascade descends into `u3lib`);
- `finalize()` — tear everything down in reverse;
- `initialize_thread()` / `finalize_thread()` — per-thread setup for multithreaded use;
- `clear()` — drop cached coefficients while keeping the databases alive;

Header	Entry point	Task	Reference
`U4irrep.h`	`dim`, `C1/C2/C3/C4`, `conjugate`	irrep dimension, Casimirs, conjugation	—
`U4basis.h`	`Zeta`, `gelfandBasis`, `physicalBasis`	inner (spin-isospin) multiplicity; basis enumeration	[1]
`U4decomposition.h`	`Eta`, `decomposition`	outer multiplicity; $A \otimes B$ series	[1]
`U4U3xU4U3.h`	`wigU4U3db`	SU(4)⊃U(3) canonical reduced CG	[1]
`U4SU2SU2xU4SU2SU2.h`	`wigU4SU2SU2db`	SU(2)×SU(2) physical CG	[1]
`U4_U6Z6Coeffs.h`	`u4u6db`, `u4z6db`	U- and Z-recoupling (three irreps)	[1]
`U4_U9Coeffs.h`	`u4u9db`	9-coefficient recoupling (four irreps)	[1]
`U4SU2SU2_to_U4U3U2U1.h`	`phys_to_cano`	physical → canonical transformation	[1]
`U4generatorsRME.h`	`RME_S`, `RME_T`, `RME_U`	reduced matrix elements of **S**, **T**, **U**	—

TABLE I: Main entry points of `u4lib`.

- `is_initialized()` — query the state.

Co-use with `u3lib`: Within `u4lib`, the lifecycle automatically cascades to `u3lib`. However, if an application also drives standalone `u3lib` directly in the same process, both libraries must be initialized at application startup and finalized only after all coefficient work is complete. Calling `finalize()` on one library while the other is still active will destroy the shared Wigner backend and crash the process.

B. Irreps: `su4::irrep` and `u4::irrep`

(`U4irrep.h`.) `su4::irrep` stores an SU(4) irrep (P_1, P_2, P_3); `u4::irrep` derives from it and adds the fourth label n_{44} (fixing the total quanta).

- **Constructors.** `su4::irrep()`, `su4::irrep(P1,P2,P3)`; `u4::irrep()`, `u4::irrep(n14,n24,n34,n44)`, `u4::irrep(P1,P2,P3)` (normalized representative), and `u4::irrep(su4::irrep, A)` (fix the total quanta A).
- **Labels.** `P1()`, `P2()`, `P3()`; `n14()`, `n24()`, `n34()`, `n44()`.
- **Dimension and invariants.** `dim()` (both classes); `C2()`, `C3()`, `C4()` — the SU(4) Casimirs of orders 2, 3, 4 (both classes, shift-invariant); `C1()` — the first-order (U(1)) Casimir = n_{14}+n_{24}+n_{34}+n_{44} (total quanta), **available on `u4::irrep` only** (SU(4) is U(4) with the U(1) trace generator removed, so `su4::irrep` has no `C1()`); `quanta()` (synonym of `C1()`).
- **Other.** `zero()`; `conjugate()` (returns (P_3, P_2, P_1)); `conjugate_phase_power()`; `normalize()` (`u4::irrep`, set $n_{44} = 0$); comparison operators and stream output; `writeBinary`/`readBinary`.

C. Gelfand states: `u4::gelfand`

(`U4gelfand.h`.) The canonical basis state with top U(4) labels $[n_{14}, n_{24}, n_{34}, n_{44}]$ and U(3)⊃U(2)⊃U(1) sublabels $[n_{13}, n_{23}, n_{33}]$, $[n_{12}, n_{22}]$, n_{11}.

- **Constructor.** `u4::gelfand(n14,n24,n34,n44,n13,n23,n33,n12,n22,n11)`.
- **Labels / validity.** `n14()` … `n11()`; `check()` (betweenness condition); `normalize()` (shift $n_{44} \to 0$).
- **Subgroup labels.** `P1()`, `P2()`, `P3()`; `Lambdax2()`, `MLambdax2()` (2× the embedded SU(2) angular momentum and its projection).
- **Conjugation.** `conjugate()`, `conjugate_phase()`.
- **Generator matrix elements.** The matrix elements of the U(4) generators E_{ij} acting on the state, returned by the 44 methods `e11()` … `e44()`, `e12/e21`, `e23_*`, `e13_*`, `e34_*`, `e24_*`, `e14_*` (and their lowering partners), where a suffix selects the target branch. The raising-action predicates `E12`, `E23_12`, …, `E14_33_22` in `U4basis.h` identify the target state.

D. Physical states: `u4::physicalvector`

(`U4basis.h`.) The $\mathrm{SU}(2)\times\mathrm{SU}(2)$ reduced spin-isospin state $|(P_1P_2P_3)\,\zeta\,S\,T\rangle$. Spin and isospin are stored **doubled**: $SS = 2S$, $TT = 2T$.

- **Constructor.** `u4::physicalvector(irrep, SS, TT)`.
- **Accessors.** `irr()`; `P1()`, `P2()`, `P3()`, `n14()` … `n44()`; `SS()`, `TT()`; `zeta()` (=`zetamax()`, the multiplicity of (S,T)).

E. Free functions

- **Multiplicities.** `u4::Zeta(irrep,SS,TT)` — the inner (spin-isospin) multiplicity $\zeta_{\max}$ of (S,T), guarded to return 0 off the admissible grid; `u4::Eta(A,B,C)` (`U4decomposition.h`) — outer multiplicity $\eta_{\max}$ of C in $A\otimes B$.
- **Bases.** `u4::gelfandBasis(irrep)` — all Gelfand states; `u4::physicalBasis(irrep)` — all physical states with nonzero ζ; `u4::findGelfand(g,gArr)` — index of a state.
- **Decomposition.** `u4::decomposition(A,B)` $\to$ (list of C, list of $\eta_{\max}$); the full Kronecker series with $\sum_C \eta_{\max}(C)\dim C = \dim A\dim B$.
- **Generator RMEs.** `RME_S`, `RME_T`, `RME_U` (`U4generatorsRME.h`) take two `physicalvector` states (bra, ket) and two `std::vector<int>` out-parameters, and return an `Eigen::MatrixXd` of shape $\zeta'_{\max}\times\zeta_{\max}$. **S** and **T** are diagonal in (S,T): $\langle\|\mathbf{S}\|\rangle = \sqrt{S(S{+}1)(2S{+}1)(2T{+}1)}$ and $\langle\|\mathbf{T}\|\rangle = \sqrt{T(T{+}1)(2T{+}1)(2S{+}1)}$; **U** is the rank-$(1,1)$ Gamow–Teller-type tensor ($\Delta S,\Delta T\in\{-1,0,+1\}$).
- **Transformation.** `u4::phys_to_cano(p,MMS,MMT,gArr)` and `u4::phys_to_cano_allM(p,gArr)` (`U4SU2SU2_to_U4U3U2U1.h`) expand a physical state (at doubled projections MMS, MMT) in the Gelfand basis; they return the canonical-state indices into `gArr=gelfandBasis(p.irr())` and an amplitude matrix with one orthonormal column per ζ.

F. Coefficient containers

(`Definitions.h`.) Every database call returns one of these structured containers; indices are zero-based and bounded by the `*max()` counts.

- `U4U3` (canonical coupling) — `rhomax()`, `etamax()`; `operator()(rho,eta)`; `data()`; `size()`, `empty()`.
- `U4SU2SU2` (physical coupling) — `zetaAmax()`, `zetaBmax()`, `zetaCmax()`, `etamax()`; `operator()(zetaA,zetaB,zetaC,eta)`; `data()`.
- `U4U6` (U-recoupling) — `etaABmax()`, `etaAB_Cmax()`, `etaBCmax()`, `etaA_BCmax()`; `operator()(etaAB,etaAB_C,etaBC,etaA_BC)`; `data()`.
- `U4Z6` (Z-recoupling) — `rhoABmax()`, `rhoAB_Cmax()`, and (reusing the container's getter names) `rhoBCmax()` $\equiv$ the AC channel, `rhoA_BCmax()` $\equiv$ the AC_B channel; `operator()(etaAB,etaAB_C,etaAC,etaAC_B)`; `data()`.
- `U4U9` (9-coefficient recoupling) — `etaACmax()`, `etaBDmax()`, `etaABCDmax()`, `etaABmax()`, `etaCDmax()`, `etaACBDmax()`; `operator()(etaAC,etaBD,etaABCD,etaAB,etaCD,etaACBD)`; `data()`.

In C++ the `U4U9` result is a view into a thread-local scratch buffer that the next database call overwrites; copy it before the next call. The others own their buffers. (The Python bindings copy `U4U9` automatically.)

G. Coefficient databases

Reached through singletons; each returns a container above.

- `u4::wigU4U3db().get_U4U3(` $n^a_{14}, n^a_{24}, n^a_{34}, n^a_{44}, n^a_{13}, n^a_{23}, n^a_{33},\ \ldots^b,\ \ldots^c$ `)` $\to$ `U4U3` — SU(4) $\supset$ U(3) reduced coupling for $a \otimes b \to c$ (each state by its U(4) labels and U(3) sublabels), indexed by (ρ, η).
- `u4::wigU4SU2SU2db().get_U4SU2SU2(` $P^a_1, P^a_2, P^a_3, SS^a, TT^a,\ \ldots^b,\ \ldots^c$ `)` $\to$ `U4SU2SU2` — physical spin-isospin reduced coupling ($SS = 2S$, $TT = 2T$), indexed by $(\zeta_A, \zeta_B, \zeta_C, \eta)$.
- `u4::u4u6db().get_U4U6(` $A, B, ABC, C, AB,\ \eta_{AB}, \eta_{AB_C},\ BC, \eta_{BC}, \eta_{A_BC}$ `)` $\to$ `U4U6` — recouples $(AB)C \leftrightarrow A(BC)$ (note that ABC precedes C).
- `u4::u4z6db().get_U4Z6(` $B, A, ABC, C, AB,\ \eta_{AB}, \eta_{AB_C},\ AC, \eta_{AC}, \eta_{AC_B}$ `)` $\to$ `U4Z6` — recouples $(AB)C \leftrightarrow (AC)B$ (note the leading B, A, exchanging the first pair).
- `u4::u4u9db().get_U4U9(` $A, B, C, D,\ AB, CD, AC, BD, ABCD,\ \eta_{AC}, \eta_{BD}, \eta_{ABCD},\ \eta_{AB}, \eta_{CD}, \eta_{ACBD}$ `)` $\to$ `U4U9`.

The multiplicity maxima are the `Eta` values for the corresponding channels, supplied in the fixed interleaved positions shown; a mismatch trips an internal assertion. In `get_U4U9`, note that the 3rd multiplicity argument η_{ABCD} corresponds to the $(AB)(CD) \to ABCD$ coupling route and the 6th (η_{ACBD}) to $(AC)(BD) \to ABCD$; swapping them trips an internal assertion. A forbidden or empty coupling returns an empty container (safe to query for shape, not to index).

A minimal C++ session:

```
u4::initialize(&argc, &argv);
// (1,0,0) S=1/2 T=1/2  x  (1,0,0) S=1/2 T=1/2  ->  (2,0,0) S=1 T=1
auto blk = u4::wigU4SU2SU2db().get_U4SU2SU2(1,0,0,1,1, 1,0,0,1,1, 2,0,0,2,2);
double x = blk(0, 0, 0, 0);     // = 1
u4::finalize();
```

H. Conventions and normalization

Phase and normalization conventions follow Ref. [1]; documented values and the generation procedure are in `test/reference/`.

- **Doubled spin-isospin.** All spin and isospin labels are doubled: $SS = 2S$, $TT = 2T$ (and $MMS = 2M_S$, $MMT = 2M_T$). `Zeta` returns 0 off the parity-correct (S, T) grid; enumerate valid states with `physicalBasis` rather than scanning it.
- **Multiplicity indexing.** Counts come from `Zeta`/`Eta`; container accessors use zero-based indices $0, \ldots, \text{max}-1$.
- **Canonical-coupling phase.** For each η block the phase and normalization are fixed by the highest-weight reduced coefficients — the orthonormal null space of the system imposed by the raising generators annihilating the highest-weight coupled state. For $\eta_{\max} > 1$ (and self-conjugate irreps) the solver fixes a valid orthonormal η basis; any alternative must be applied consistently to every η-carrying coefficient. Conjugation of a Gelfand state satisfies $C\,|g\rangle = (-1)^{n_{14}+n_{34}-n_{13}-n_{23}-n_{33}-n_{12}-n_{22}-n_{11}}\,|g.\text{conjugate}()\rangle$ (`gelfand::conjugate_phase()`).
- **Physical → canonical phase.** A physical state $|R\,\zeta\,S\,M_S\,T\,M_T\rangle$ ($R = (P_1, P_2, P_3)$) is expanded in canonical Gelfand states. The highest-weight state $|R\,\zeta\,S\,S\,T\,T\rangle$ is the null space of $S_+ = E_{13}+E_{24}$ and $T_+ = E_{12}+E_{34}$; S_- and T_- fix the lower-M phases within a ζ multiplet. For $P_1 \neq P_3$ the construction preserves ζ; for a self-conjugate irrep ($P_1 = P_3$), conjugation acts as a projection-independent symmetric matrix D ($D^2 = \mathbb{1}$) that may mix ζ (e.g. $(P_1, P_2, P_3) = (1, 1, 1)$, $S = T = 1$).
- **Generator RME factor.** `RME_S`/`RME_T`/`RME_U` carry an extra $\sqrt{(2S'+1)(2T'+1)}$ relative to the Wigner–Eckart reduced element.
- **Assertions are on.** Invalid labels or out-of-range indices abort in C++; the Python bindings guard database access and raise instead.

IV. PYTHON BINDINGS

The module `u4lib` mirrors the C++ API one-to-one; build it with `make python` and the interpreter whose ABI matches the extension (keep `u3lib`'s `python/` importable, since `u4lib` imports it). Irreps (`su4.irrep`, `irrep`), `gelfand`, `physicalvector`, the free functions `Eta`, `Zeta`, `decomposition`, `gelfandBasis`, `physicalBasis`, the physical-generator RMEs `RME_S`/`RME_T`/`RME_U`, the transformation `phys_to_cano`/`phys_to_cano_allM`, and the five database accessors are all exposed with the same names and semantics as Sec. III. The structured containers provide `__call__` (multiplicity-indexed), flat `data()` as a NumPy array, and a printing method; database access before `initialize()` or after `finalize()` raises `RuntimeError` rather than aborting.

```
import u4lib as u4
u4.initialize()
try:
    blk = u4.wigU4SU2SU2db().get_U4SU2SU2(1,0,0,1,1, 1,0,0,1,1, 2,0,0,2,2)
    x = blk(0, 0, 0, 0)                  # (zetaA,zetaB,zetaC,eta) = 1
finally:
    u4.finalize()
```

A worked introduction covering irreps, bases, coupling, and recoupling is provided as a Jupyter notebook, `python/tutorial.ipynb`; ABI and toolchain details are in `python/BUILD_NOTES.md`.

V. AI-ENABLED INTERFACE

`u4lib` ships a repository-local catalog of *agent skills* under `agents/`: an index (`agents/AGENTS.md`), shared runtime instructions, and one skill per capability. Each skill states its inputs, the exact library call, the convention, and a built-in numerical check. A code-executing AI assistant pointed at the catalog carries out the requested computation using the configured local library; no network service is involved.

It is important to be precise about what the agent does and does not do. **The agent performs no physics and no mathematics itself.** It does not derive or approximate coupling coefficients, Casimirs, or matrix elements. Its role is *orchestration*: from a natural-language request it selects the matching skill, constructs valid inputs in the library's conventions, invokes the compiled C++/Python routine, and reports the returned numbers with their labels and the check that was run. All numerical work is done by the library; the agent is an interface that routes a request to the correct tool and formats the result, and it declines or asks for clarification when a request is ill-posed (for example, a request for the first-order Casimir of an SU(4) irrep, which is not defined).

Table II lists each skill with a representative prompt; the parenthetical results were checked against the library, so each prompt doubles as a self-test. Spin and isospin use the doubled labels $SS = 2S$, $TT = 2T$.

VI. BUILD AND INSTALLATION

`u4lib` is header-only and layers on `u3lib` (a prerequisite); user programs include the headers in `include/` directly. The source code is hosted publicly at https://github.com/Deep-SANE/u4lib. Dependencies and compiler flags are resolved uniformly via `make config` (or `./config/configure.sh`), which inspects the local environment, locates `u3lib`, and writes `config/config.generated.mk` without requiring manual environment variables. The main make targets are `make` (builds example CLI utilities → `bin/`), `make test` (rebuilds and runs the regression test suite), and `make python` (compiles the pybind11 extension `python/u4lib.so`). The regression tests check dimensions, Gelfand structure, SU(2)×SU(2) branching, canonical/physical coupling orthonormality across ζ and η, conjugation identities, and documented analytic values; numerical agreement is tolerance-based.

All core external dependencies are provided by `u3lib`:

- `u3lib` (prerequisite) — provides the base group-theoretic infrastructure, shared utilities, and the libwignernj SU(2) Wigner backend.
- **Null-space solver** (selected via `NULLSPACE_SOLVER=qr|sns`, default `qr`):
  - Dense QR (default) — dense QR null-space solver via Eigen3; requires no external sparse solver.
  - `sns` (https://gitlab.fit.cvut.cz/krastale/dp-sparse-nullspace) — sparse null-space solver enabled via `USE_SNS=1`.
- Eigen3, Boost (headers), OpenMP, and fmt — inherited from `u3lib`.

Skill	Example prompt (expected result)
`dim`	"Dimension of the SU(4) irrep $(1,0,1)$ – equivalently the U(4) irrep $[2,1,1,0]$?" (15, the adjoint; shift-invariant)
`casimirs`	"C_2, C_3, C_4 for the SU(4) adjoint $(1,0,1)$, and C_1, C_2, C_3, C_4 for the U(4) irrep $[2,1,1,0]$." (SU(4): 16, 0, 72; U(4): C_1=4, 16, 0, 72 – C_1 is U(4)-only)
`cano-list`	"Enumerate the canonical U(4) Gelfand basis of $(1,0,0)$ with each state's weights and subgroup labels." (4 states = dim)
`phys-list`	"List the physical spin-isospin basis (ζ, S, T) of $(2,0,0)$ and verify the dimension sum rule." ((S,T)=(1, 1), (0, 0), each ζ=1; $\sum \zeta(2S+1)(2T+1)$=10)
`cano-generators`	"Act with E_{12} on the Gelfand state $\lvert 2,0,0,0;2,0,0;2,0;1\rangle$ of $(2,0,0)$; give the resulting state and matrix element." ($\sqrt{2}$)
`phys-generators`	"Compute $\langle(2,0,0)\,S{=}1\,T{=}1\|\|\mathbf{S}\|\|(2,0,0)\,S{=}1\,T{=}1\rangle$ and the diagonal $\langle\|\|\mathbf{U}\|\|\rangle$." ($\sqrt{18} \approx 4.243$ and 3)
`phys-to-cano`	"Express the highest-weight $\lvert(2,0,0)\,\zeta{=}1\,S{=}1\,T{=}1\rangle$ in the canonical Gelfand basis." (a single state, amplitude 1)
`inner-mult`	"Spin-isospin multiplicity ζ of (S,T)=(1, 1) in the adjoint $(1,0,1)$?" (1)
`outer-mult`	"Outer multiplicity η of $(1,0,1)$ in $(1,0,1)\otimes(1,0,1)$?" (2)
`decomposition`	"Full decomposition of $(1,0,0)\otimes(1,0,0)$ with outer multiplicities; check the dimension sum rule." ($(2,0,0)+(0,1,0)$, $\sum \eta\cdot\dim$ =16)
`cano-coupling`	"SU(4)⊃U(3) reduced coupling for the highest weights of $(1,0,0)\times(1,0,0)\to(2,0,0)$; verify it equals 1."
`phys-coupling`	"Spin-isospin coupling $(1,0,0)\,S{=}\frac{1}{2}T{=}\frac{1}{2}\times(1,0,0)\,S{=}\frac{1}{2}T{=}\frac{1}{2}\to(2,0,0)\,S{=}1T{=}1$; verify it equals 1."
`u6recoupling`	"U-recoupling for $(1,0,0)\cdot(1,0,0)\cdot(1,0,0)$ through $(2,0,0)$ to $(3,0,0)$." (1)
`z6recoupling`	"Z-recoupling between the AB–C and AC–B routes for $(1,0,0)\cdot(1,0,0)\cdot(1,0,0)\to(3,0,0)$." (1)
`u9recoupling`	"9-coefficient recoupling for four fundamentals $(1,0,0)$ coupling to $(4,0,0)$." (1)
`conjugation`	"Conjugate the SU(4) irrep $(2,0,0)$ and a Gelfand state of it, reporting the phases." (irrep $\to(0,0,2)$, phase power 3)

TABLE II: The `u4lib` agent skills with a representative prompt for each.

- pybind11 (Python bindings) — https://github.com/pybind/pybind11.

`u4lib` ships as an independent, self-contained repository with a uniform `make config`/`make`/`make test`/`make install` interface and no required environment variables.

Acknowledgments

This work is dedicated to the memory of Jerry P. Draayer (who passed on September 13, 2026), whose pioneering contributions to algebraic nuclear physics and Wigner's supermultiplet model inspired this framework. This library implements the U(4) coupling and recoupling procedure of Ref. [1]. The authors also gratefully acknowledge Feng Pan and Michela Negro for their valuable discussions and contributions.

[1] P. Dang, J. P. Draayer, F. Pan, T. Dytrych, D. Langr, D. Kekejian, K. S. Becker, and N. Thompson, "Coupling and Recoupling Coefficients for Wigner's U(4) Supermultiplet Symmetry," Eur. Phys. J. Plus **139**, 933 (2024).